

CS683 PA-1, Task 1: Hardware-Conscious 2D Convolution

Tasks 1A–1D — Loop Reordering/Unrolling, Tiling, SIMD, and the Combined Kernel

Methodology

All measurements are single-channel `float32` 2D convolution (“same” mode, zero-padded, cross-correlation, kernel not flipped), built with the assignment’s pinned flags:

```
(-std=c++17 -O2 -fno-tree-vectorize -mavx2 -mfma)
```

Hardware. A 13th Gen Intel Core i5-13420H, a hybrid P/E-core CPU (4 P-cores + 8 E-cores).

Correctness. Every kernel used to produce a number in this report — the five graded stages and the extra bench-only variants (128/256-bit width study, the `tile×width×unroll` combo grid) — was checked against `conv_naive` directly on this machine, across multiple sizes, both K values, and multiple random seeds. All passed.

Speedup. $\text{Speedup} = \frac{\text{time}_{\text{naive}}}{\text{time}_{\text{optimized}}}$, equivalently the ratio of achieved GFLOP/s since the FLOP count ($2K^2HW$) is fixed for a given workload.

Instruction counts. `perf`’s `instructions` event counts the whole measured process, which calls the kernel `warmup(2) + reps(20) = 22` times (one-time setup is negligible by comparison); all instruction counts quoted below are that total divided by 22, i.e. per single invocation.

1 Task 1A — Loop Reordering and Unrolling

`conv_naive` accumulates all K^2 taps for one output pixel into a single scalar before moving to the next pixel: `acc += in[...] * ker[...]` inside a $K \times K$ loop. Every iteration of that inner loop depends on the previous one (it reads and writes the same `acc`), so the loop runs at FMA *latency* (~ 4 cycles) rather than FMA *throughput* (2/cycle) — the CPU is stalled waiting for one accumulation to finish before it can start the next, even though the hardware could in principle run two FMAs per cycle. `conv_reorder` removes this dependency chain by hoisting the tap loop outside: for a fixed tap (k_y, k_x) , the weight is loop-invariant and the update `out[oy][ox] += w * in[oy+ky][ox+kx]` touches a *different* output element on every iteration, so consecutive iterations have nothing to wait on. `conv_unroll` takes the same reordered loop nest and unrolls the innermost (column) loop 8-wide, giving the compiler 8 independent loads/FMAs/stores per iteration.

$H=W$	naive instr/call	reorder instr/call	unroll instr/call	naive time (ms)
512	23,830,248	17,044,775	9,046,332	2.283
1024	94,829,763	67,648,897	35,646,511	7.873
2048	378,844,454	269,987,885	142,060,542	31.500

Table 1: Per-call instruction count, $K=3$. Reordering already removes address recomputation across taps; unrolling removes it again per 8-wide block, roughly halving the count a second time.

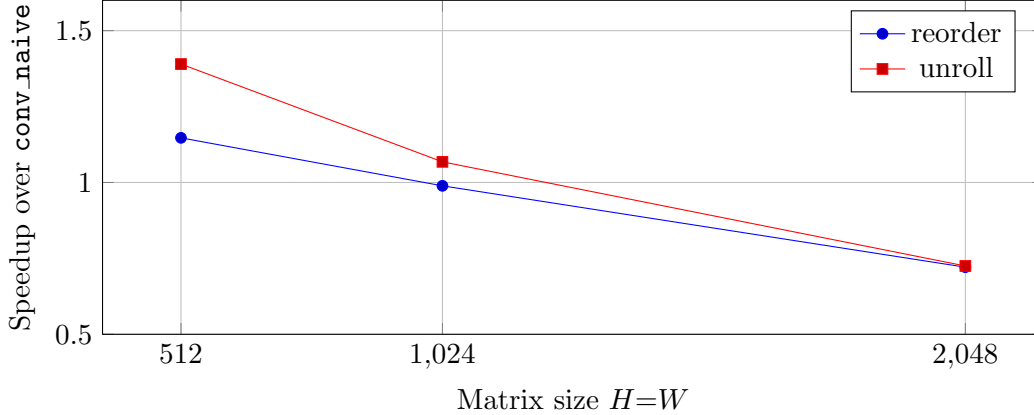


Figure 1: Speedup vs. matrix size, $K=3$. Both techniques *lose* to naive at 2048×2048 despite executing far fewer instructions.

Q1: What motivated the changes?

The naive loop carries a true dependency chain through its single accumulator, capping throughput at FMA latency instead of FMA throughput. That is a pure instruction-level parallelism (ILP) problem, independent of any memory effect, and it is fixable purely by changing loop order and exposing independent work.

Q2: What considerations were taken into account?

(1) *Aliasing*: `in` and `out` are distinct buffers, but the compiler cannot prove that without `__restrict`, and would otherwise reload `out` on every store in case it aliases `in`. (2) *Correctness under reordering*: summing the same K^2 products in a different order changes floating-point rounding, so correctness is checked with a tolerance, not bit-exactness; observed error is $\sim 10^{-6}$. (3) *Unroll factor and loop-invariant hoisting*: 8, chosen jointly for $W \bmod 8=0$ (no scalar tail) and for direct compatibility with the AVX2 register width used two stages later. The tap weight and row base pointers are computed once per tap/row, not once per pixel.

Q3: How effective are the changes, and which metric?

Wall-clock time is the metric that matters, *not* instruction count. At 2048×2048 , `reorder` executes 29% fewer instructions than naive (269.99M vs. 378.84M) and `unroll` executes 62% fewer (142.06M). The reason is that reordering trades the naive’s single pass with a tiny, cache-resident working set for K^2 full passes over the entire 16MB output image. This is confirmed directly with LLC-miss counters (Table 2): naive’s LLC misses are ~ 266 K at this size, while `reorder`’s are

$\sim 55.6\text{M}$ – two orders of magnitude more real DRAM traffic. At 512×512 , the working set is small enough that both passes stay cache-resident, so the ILP win is realized.

2 Task 1B — Cache Tiling

L1-D cache size

48 KiB per P-core, 12-way associative, 64 B lines.

Baseline (naive) MPKI

0.78, 0.76 and 0.87 L1-D misses per 1000 instructions at $H=W=512, 1024, 2048$ ($K=3$). This is flat and near zero at every size, because naive only ever keeps 3 rows of the padded input alive at a time — about $(W+2) \times 3 \times 4\text{B}$.

Tiled implementation

`conv_tile` changes the loop order to `(tile) → (tap  $k_y, k_x$ ) → (row, col inside tile)`. All K^2 taps are applied to one tile before moving to the next, so the tile and its halo should stay in a fast cache level for all K^2 passes. Tile sizes 16, 32, 64, 128, 256, 512 (square) were tested at every image size.

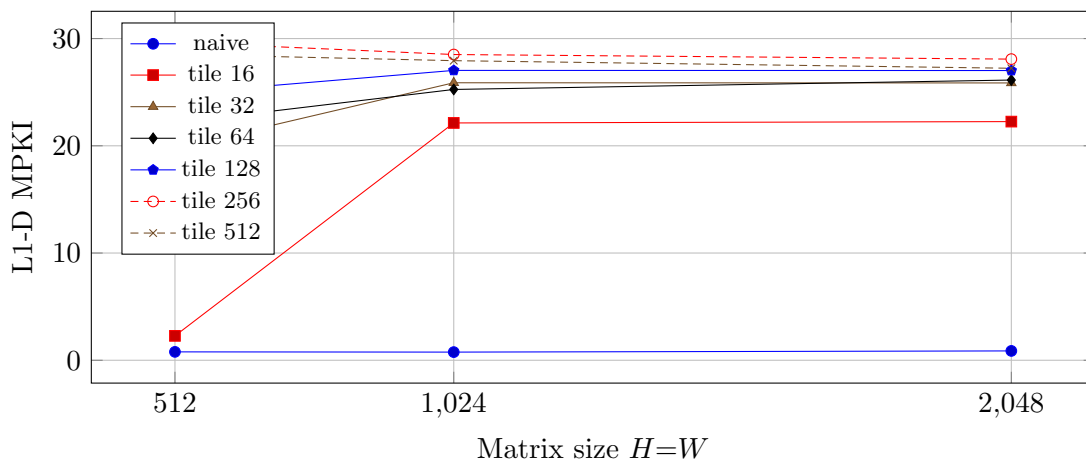


Figure 2: L1-D MPKI vs. matrix size, by tile size ($K=3$)

Where the misses actually go: L2 and LLC counts

L1-D MPKI on its own does not tell us whether a miss was served quickly by L2/L3 or had to go all the way to DRAM. So we also counted L2 misses (`l2_rqsts.miss`) and LLC misses (`llc-load-misses`) on the graded 2048×2048 , $K=3$ run.

Tile 16×16 has about 12x as many L1 misses as naive (89.1M vs. 7.5M), which is expected: every tile re-reads its halo once per tap, and there are a lot of tiles. But its L2 and LLC misses stay at naive’s level (13.4M and 341K). Those extra L1 misses are being caught by L2/L3 instead of going to DRAM, because a 16×16 tile plus halo is small enough to stay in cache for all K^2 passes.

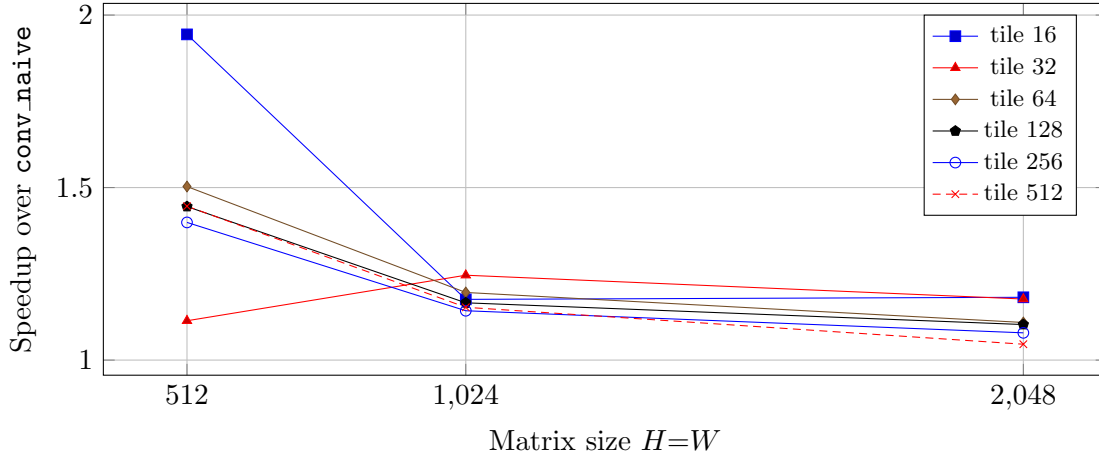


Figure 3: Speedup vs. matrix size, by tile size ($K=3$). Best tile size: 16×16 at 512×512 and 2048×2048 , 32×32 at 1024×1024 (margins are small, 1–5%).

Table 2: L1/L2/LLC miss counts, 2048×2048 , $K=3$ (single-run **perf** counts, not divided by call count — used here only as a ratio).

kernel	L1-D misses	L2 misses	LLC misses
naive	7,499,850	13,451,231	266,194
tile 16×16	89,082,717	13,433,313	340,922
tile 512×512	85,891,539	112,906,336	5,332,681

Tile 512×512 is the opposite case. Its tile no longer fits, so L2 misses jump to 112.9M and LLC misses to 5.3M. So tiling does not reduce how many misses happen — it changes **where** they are served, and that is why it is faster.

Q1: MPKI change, naive $\rightarrow$ tiled

L1-D MPKI *goes up*, from 0.78–0.87 for naive to roughly 2–30 for the tiled version, i.e. 2–40x worse depending on tile size.

Judged on MPKI alone this looks like a regression, but Table 2 shows it is not. Nearly all the extra misses are served by L2/L3 rather than DRAM, and the run time still improves (Q3).

Q2: How MPKI varies with tile size and matrix size

Across tile sizes, at one image size: MPKI goes up as the tile gets bigger. The tile and its halo stop fitting in the 48 KiB L1, and then stop fitting in L2. The L2/LLC counts in Table 2 show the same trend one level down.

Across image sizes, for one tile size: this one is less obvious. A fixed tile shape touches the same number of bytes no matter how big the image is, so MPKI should not move — but tile 16×16 goes from 2.27 at 512×512 to about 22.1–22.3 at 1024 and 2048.

The padded row stride $W+2p$ is the likely reason. This L1 has 64 sets (48 KiB / 64 B / 12-way), and a cache line’s address decides which set it lands in. When W changes, the gap in bytes between one row of the tile and the next changes too, and for some values of W several rows of the tile end up mapped to the same set. They then evict each other even though there is free space elsewhere

in the cache. This is a known effect on x86 for strides that are close to a power of two, and it fits tile 16 being hit hardest, since tile 32 and 64 have less regular strides relative to 64 sets.

Q3: Did tiling achieve speedup?

Yes, at every size and tile shape tested: 1.05x–1.94x. The best case is 1.94x at 512×512, and the graded workload (2048×2048, $K=3$) reaches 1.18x with the best tile size, 16×16.

The reason is the effect shown in Table 2: moving misses off the DRAM path is a straight latency win, even though the instruction count is unchanged.

The limiting factor is that tiling only fixes the memory side. Each of the K^2 passes over a tile is still a plain scalar loop, one element at a time, so the compute cost puts a ceiling on the speedup. Adding SIMD is the fix for that.

3 Task 1C — SIMD (AVX2)

`conv_simd` keeps the same loop structure as `conv_reorder`. The inner 8-element block is replaced by a single `_mm256_fmadd_ps`, with the tap weight copied into all 8 lanes once per tap using `_mm256_set1_ps`. W is always a multiple of 8, so there is no leftover scalar tail to handle.

Instruction counts, naive vs. SIMD

$H=W$	naive instr/call	<code>conv_simd</code> (256-bit) instr/call	reduction
512	23,830,248	2,767,821	8.61x
1024	94,829,763	10,556,824	8.98x
2048	378,844,454	41,724,996	9.08x

Table 3: $K=3$. The reduction stays at about 8.6–9.1x at every image size, which is what we expect when the saving comes purely from processing 8 elements at a time.

Speedup vs. matrix size and SIMD width

Q1: Instruction count change, naive → SIMD

The instruction count drops by 8.6–9.1x, and the factor barely changes with image size ($K=3$). So the saving is set almost entirely by the 8-wide data path.

Q2: Speedup achieved — what helps and what limits it

Yes, but it shrinks badly as the image grows: 3.19x at 512×512, down to 1.23x at 2048×2048 — even though the instruction count stays about 9x lower at every size.

What helps at small sizes: 9x fewer instructions, fused multiply-add (one instruction and one rounding step instead of two, matching the `-mfma` flag), and the independent per-element work inherited from `conv_reorder`.

What limits it at 2048×2048 is DRAM bandwidth, the same wall seen in Task 1A. `conv_simd` is still untiled, so it sweeps the whole 16 MB image K^2 times.

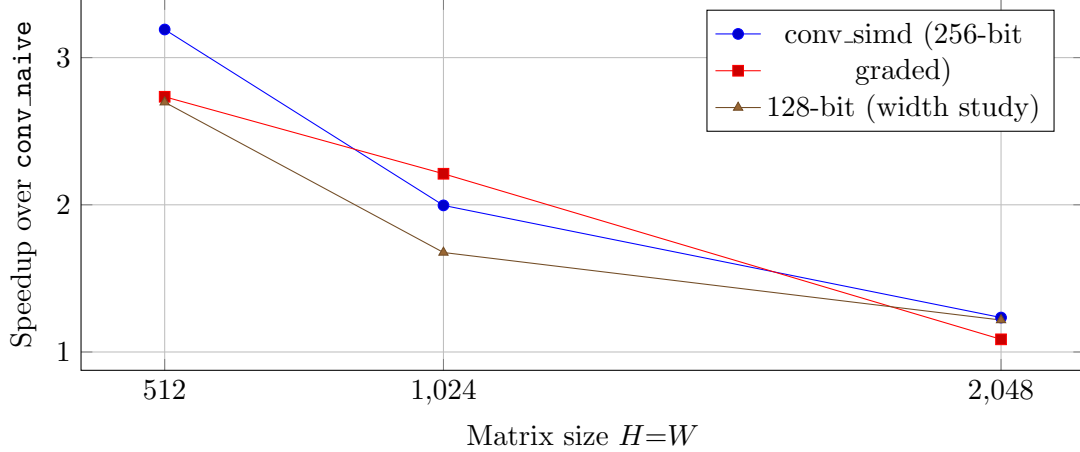


Figure 4: Speedup vs. matrix size and SIMD width, $K=3$. The two 256-bit lines are the same algorithm run through two paths and agree within run-to-run noise, which cross-checks the harness. All widths fall to about 1.1–1.2x at 2048×2048 .

Q3: Which SIMD intrinsics, and why

- `_mm256_set1_ps` — copies the tap weight into all 8 lanes. Done once per tap, hoisted out of the row/column loops.
- `_mm256_loadu_ps` / `_mm256_storeu_ps` — unaligned load and store.
- `_mm256_fmadd_ps` — fused multiply-add: one instruction and one rounding step per tap per 8 outputs, matching the `-mfma` flag.
- `_mm256_mul_ps` — used for the first tap only, which writes its result rather than adding to it.

4 Task 1D — Sabka Saath Sabka Vikaas

`conv.optimized` applies all three earlier techniques, in the order they depend on each other:

1. **Reordering** first — it gives the loop shape the other two need: independent per-element work, walking memory in unit stride.
2. **Tiling** next — it keeps the K^2 passes in cache, which is the step that removes the bandwidth bottleneck seen in Tasks 1A and 1C.
3. **AVX2 with a 2-vector (16-column) unroll** last — it turns the headroom the first two opened up into actual throughput.

The tile shape is $64 \text{ rows} \times 256 \text{ columns}$. This was checked on this machine by timing it against square tiles from 16 to 1024 in the same run: 64×256 wins every time, taking 7.27 ms at 2048×2048 against 8.2–10.3 ms for the next best (128×128). It is not a leftover default. It differs from Task 1B’s best tile size because SIMD and unrolling lower the cost per element, which shifts the balance the tile size is trading off against.

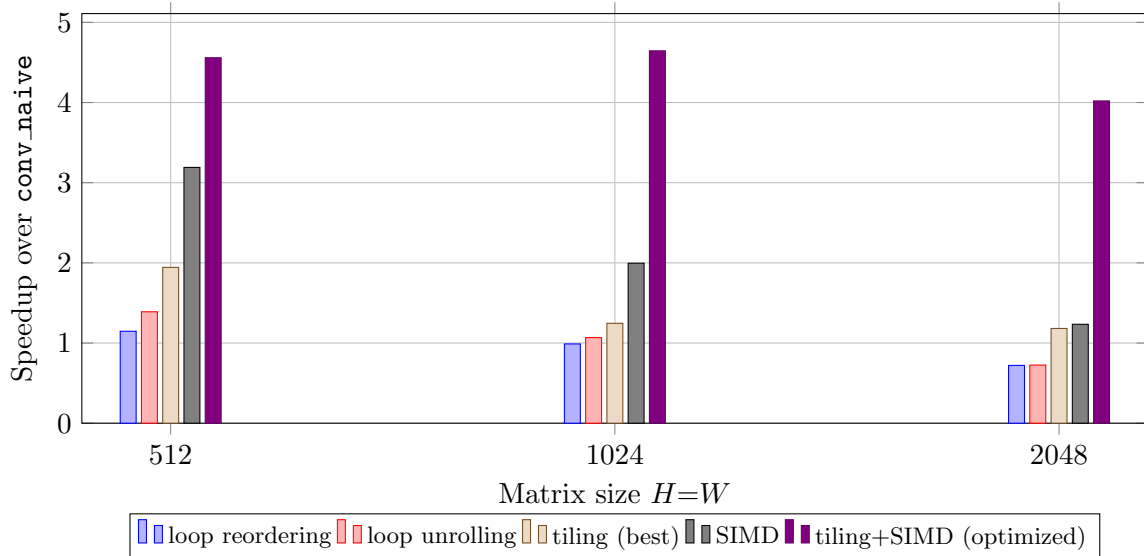


Figure 5: Best speedup per technique vs. matrix size, $K=3$ (cf. Fig. 1.1 in the assignment). Each technique on its own stays between 1x and 3.2x and gets worse as the image grows, dropping to 1x or below. The combined kernel holds 4.0–4.6x at every size.

Table 4: Individual best speedups vs. the combined kernel, $K=3$. The last column is a simple reference, $1 + \sum(\text{speedup}_i - 1)$ — what you would get if the gains just added up. It is not a real speedup law, only a yardstick.

$H=W$	reorder	unroll	tile	SIMD	optimized	additive ref.
512	1.15x	1.39x	1.94x	3.19x	4.56x	4.67x
1024	0.99x	1.07x	1.25x	2.00x	4.64x	2.30x
2048	0.72x	0.72x	1.18x	1.23x	4.02x	0.86x

Demonstrating synergy

At 512×512 the combined kernel roughly matches the additive reference (4.56x vs. 4.67x). Every technique already works on its own at this size, so there is not much extra to gain from combining them.

At 1024×1024 , and far more so at 2048×2048 , the combined kernel beats the reference by a wide margin. At 2048×2048 no single technique gets past 1.23x, two of them are below 1x (i.e. they make things worse on their own), and the additive reference is only 0.86x — yet `conv_optimized` reaches 4.02x.

So the techniques are clearly not separate gains that can be added together. Reordering and SIMD only pay off once tiling has removed the DRAM bandwidth bottleneck that made them harmful by themselves at this size. “Sabka saath, sabka vikaas” — together each technique opens up room for the others, while alone several of them actually make the kernel slower at scale.

Autograder Result on This Machine

All five graded stages are correct (40/40 correctness points). `conv_optimized` reaches 4.02–4.65x at $K=3$ across sizes (4.41x on the harness’s own graded run, median-of-7), landing in the 4x–6x

tier (30/60 speedup points), for a total of **70/100**.

A $K=5$ results (not graded, for reference)

$H=W$	reorder	unroll	tile	SIMD	optimized	additive ref.
512	1.04x	1.14x	1.58x	2.98x	5.12x	3.74x
1024	1.08x	1.14x	1.41x	2.23x	5.74x	2.88x
2048	0.66x	0.66x	1.24x	1.13x	4.66x	0.69x

Table 5: Same structure as the $K=3$ table above; the synergy finding is identical (and slightly stronger: optimized reaches 4.66–5.74x here, vs. an additive reference that falls below 1x at 2048×2048).